

Объектно ориентированное программирование на C#

Тема 24. Практика. Работа с JSON

Задание 1: Сериализация объекта в JSON.

Создайте класс Person с полями, представляющими информацию о человеке (имя, фамилия, возраст, адрес, e-mail, дата рождения).

Создайте экземпляр этого класса, заполните его данными.

Используйте библиотеку System.Text.Json для сериализации объекта Person в формат JSON.

Сохраните JSON-строку в файл.

Задание 2: Сериализация коллекции объектов.

Создайте класс Product с полями, представляющими информацию о продукте (название, цена, описание и т.д.).

Создайте список, содержащий несколько объектов класса Product.

Используйте System.Text.Json для сериализации списка продуктов в JSON-массив.

Сохраните JSON-массив в файл.

Задание 3: Чтение из JSON и десериализация.

Используйте JSON-файл из предыдущего задания с данными о продуктах.

Считайте содержимое JSON-файла в виде строки.

Десериализуйте JSON-строку в список объектов класса Product с использованием System.Text.Json.

Задание 4: Сериализация с настройками.

Создайте класс Order с различными полями для заказа (номер заказа, дата, товары и т.д.).

Настройте опции сериализации System.Text.Json для исключения определенных полей или для управления форматированием.

Создайте экземпляр класса Order, сериализуйте его с использованием настроенных опций и сохраните в файл.

Задание 5. * Десериализация готового файла.

На основании JSON файла сформируйте классы.

Задача 2.

```
using System.Text.Json;

class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string Description { get; set; }
    // Другие поля, если необходимо

    public Product(string name, decimal price, string description)
    {
        Name = name;
        Price = price;
        Description = description;
    }
}

class Program
{
    static void Main()
    {
        // Создаем список продуктов
        List<Product> products = new List<Product>
        {
            new Product("Мобильный телефон", 499.99m, "Смартфон с отличной камерой."),
            new Product("Ноутбук", 999.99m, "Легкий и мощный ноутбук для работы и развлечений."),
            new Product("Наушники", 149.99m, "Беспроводные наушники с хорошим звуком.")
        };

        // Сериализуем список продуктов в формат JSON
        string json = JsonSerializer.Serialize(products, new JsonSerializerOptions
        {
            WriteIndented = true,
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
            NumberHandling = JsonNumberHandling.AllowReadingFromString,
            DictionaryKeyPolicy = JsonNamingPolicy.CamelCase,
        });
    }
}
```

```
Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    });

    // Сохраняем JSON-массив в файл
    File.WriteAllText("products.json", json);

    Console.WriteLine("Сериализация завершена и JSON-массив
сохранен в файле 'products.json'.");
}
}
```

Задание 5.

```
using System;
using System.Text.Json;

class Program
{
    static void Main()
    {
        string jsonString = @"{
            ""employee"": {
                ""firstName"": ""John"",
                ""lastName"": ""Doe"",
                ""age"": 35,
                ""email"": ""johndoe@example.com"",
                ""phone"": ""+1 (555) 123-4567"",
                ""address"": {
                    ""street"": ""123 Main Street"",
                    ""city"": ""Anytown"",
                    ""state"": ""CA"",
                    ""zipCode"": ""12345""
                },
                ""projects"": [
                    {
                        ""name"": ""Project A"",
                        ""startDate"": ""2023-01-15"",
                        ""endDate"": ""2023-05-30"",
                        ""status"": ""In Progress""
                    },
                    {
                        ""name"": ""Project B"",
                        ""startDate"": ""2023-03-10""
```

```

        "endDate": "2023-08-20",
        "status": "Completed"
    }
}

},
"company": {
    "name": "TechCorp Inc.",
    "address": {
        "street": "456 Tech Road",
        "city": "Techville",
        "state": "CA",
        "zipCode": "54321"
    },
    "employeesCount": 5000,
    "revenue": 1000000000
}
};

// Десериализация JSON-объекта в объект C#
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true // Регистр букв в
свойствах не учитывается
};

var data = JsonSerializer.Deserialize<Root>(jsonString,
options);

// Вывод данных на экран
Console.WriteLine($"Имя сотрудника: {data.Employee.FirstName}
{data.Employee.LastName}");
Console.WriteLine($"Возраст: {data.Employee.Age}");
Console.WriteLine($"Email: {data.Employee.Email}");
Console.WriteLine($"Проекты:");
foreach (var project in data.Employee.Projects)
{
    Console.WriteLine($" - {project.Name}, Статус:
{project.Status}");
}
Console.WriteLine($"Название компании: {data.Company.Name}");
Console.WriteLine($"Выручка компании: {data.Company.Revenue}");
}
}

```

```
class Root
{
    public Employee Employee { get; set; }
    public Company Company { get; set; }
}

class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }
    public Address Address { get; set; }
    public Project[] Projects { get; set; }
}

class Address
{
    public string Street { get; set; }
    public string City { get; set; }
    public string State { get; set; }
    public string ZipCode { get; set; }
}

class Project
{
    public string Name { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    public string Status { get; set; }
}

class Company
{
    public string Name { get; set; }
    public Address Address { get; set; }
    public int EmployeesCount { get; set; }
    public long Revenue { get; set; }
}
```